PRACTICAL DATA SCIENCE WITH PYTHON

Pandas: Combining, Joining and Adding Data

A Practical Guide to Data Integration, Dataset Manipulation and Aggregation

From Individual Records to Integrated Datasets

Understanding `merge()`, `join()`, `concat()`, column assignment, row insertion and `groupby()`.

An Applied Learning Article

Python Programming • Data Analysis • Data Science

All examples use simulated taxpayer and Electronic Billing Machine (EBM) invoice data.

By Jean Baptiste HABANABAKIZE

Contents

1 Introduction **3**

2 The Fundamentals **3**

2.1 Adding columns: horizontal combination 4

2.2 Adding rows: vertical combination 4

2.3 Grouping: summarising existing rows 4

3 Preparing the Example Datasets **4**

3.1 Importing Pandas 5

3.2 Creating the invoice table 5

3.3 Creating the taxpayer table 5

3.4 How the two tables relate 5

4 MERGE: Combining Tables on a Common Column **6**

4.1 Inner join: keep only matches 6

4.2 Left join: keep everything from the left table 7

4.3 Right join: keep everything from the right table 7

4.4 Outer join: keep everything from both tables 8

4.5 Comparing the four joins 8

4.6 A picture of the four joins 9

4.7 A caution about duplicate keys 9

5 CONCAT: Stacking Tables by Rows or Columns **9**

5.1 Stacking rows: axis=0 10

5.2 Gluing columns: axis=1 10

5.2.1 What if `merge()` had been used instead? 11

6 JOIN: Combining DataFrames Using Their Indexes **12**

6.1 What a DataFrame index actually is 13

6.2 A working join 13

6.3 Join types 13

7 Adding New Columns **14**

7.1 Direct assignment 14

7.2 Calculated assignment 14

7.3 Using `assign()` 15

7.4 Using `insert()` 15

7.5 Quick comparison 16

8 Adding New Rows **16**

8.1 A single new row with `loc` 16

8.2 Multiple rows with `concat()` 17

9 GROUPBY: Grouping and Summarising **17**

9.1 A small example 18

9.2 Total sales per taxpayer 18

9.3 Grouping by more than one column 18

9.4 Why `reset_index()` matters 18

9.5 A handful of other aggregations 19

10 Other Frequently Used Operations **19**

11 Comparing the Main Combination Operations **19**

12 End-to-End Example: Monthly EBM Analysis **20**

12.1 Step 1 — keep only completed invoices 20

12.2 Step 2 — derive the invoice month 20

12.3 Step 3 — group by taxpayer and month 21

12.4 Step 4 — inspect the result 21

12.5 Step 5 — enrich with registration data 21

12.6 What the analyst can do next 21

13 Common Mistakes to Avoid **22**

13.1 Using `concat()` when you meant `merge()` 22

13.2 Choosing the wrong join type 22

13.3 Treating `NaN` as zero 22

13.4 Ignoring duplicate keys 22

13.5 Confusing `count()` and `size()` 22

13.6 Forgetting index alignment in `concat(axis=1)` 22

14 Practice and Self-Assessment **23**

14.1 Multiple-choice questions 23

14.2 Practical exercise 25

14.3 Suggested answers 25

15 Conclusion **29**

1. Introduction

Real datasets rarely arrive in one neat file. Most of the time, the information you need is scattered across departments, databases, spreadsheets and the occasional exported report. A tax office, for instance, may keep taxpayer registration details in one system and electronic invoices in another. An education project may store schools, students, teachers and examination results in separate tables. Each dataset describes a slice of the same reality, but they are not yet in a form you can analyse together.

Collecting these records is only half the job. The real work is combining them correctly: without dropping rows that matter, without duplicating records by accident, and without creating false relationships between identifiers that only look similar.

Pandas gives us several operations for exactly this task [1, 2]. The ones covered in this article are:

- `merge()` — combine datasets using matching columns or keys.
- `join()` — combine datasets using their indexes.
- `concat()` — stack datasets vertically or side-by-side.
- Column assignment, `assign()` and `insert()` — add new columns.
- `loc` and `concat()` — add new rows.
- `groupby()` — group rows and compute summaries.

Learning objectives

By the end of this article you should be able to:

1. Explain the difference between adding columns and adding rows.
2. Combine datasets with `pd.merge()`.
3. Choose correctly between inner, left, right and outer joins.
4. Use `concat()` with `axis=0` and `axis=1`.
5. Explain why `join()` depends on the DataFrame index.
6. Add columns using assignment, `assign()` and `insert()`.
7. Add rows using `loc[]` and `concat()`.
8. Summarise data with `groupby()`.
9. Pick the right operation for a given analysis problem.

2. The Fundamentals

Before reaching for any Pandas method, it helps to name the three things we actually do to data. Everything else in this article is one of them in disguise.

2.1 Adding columns: horizontal combination

Adding a column means introducing a new variable into an existing table. Suppose you already have taxpayer identification numbers and sales figures. You may want to attach the district and sector that belong to each taxpayer. The result is a wider table with the same number of rows.

TIN	Sales	District	Sector
101	500	Kigali	Retail
102	700	Huye	Tech
103	900	Rubavu	Agriculture

2.2 Adding rows: vertical combination

Adding a row means appending a new observation to an existing table. Imagine January and February invoices arrived as two separate files. Stacking them gives one taller table with the same columns.

TIN	Sales
101	500
102	700
103	900
104	400

2.3 Grouping: summarising existing rows

Grouping is neither horizontal nor vertical. It reorganises rows by a shared value and collapses each group into a summary — a total, an average, a count. Several invoices that belong to the same taxpayer, for example, collapse into that taxpayer's total sales.

TIN	Total Sales
101	1,200
102	900

Three ideas to hold in mind

Horizontal Adding or combining columns.

Vertical Adding or combining rows.

Aggregation Grouping rows and computing summary statistics.

Once you can classify a task as one of these three, the right Pandas method almost selects itself.

3. Preparing the Example Datasets

We will work with a deliberately small tax-administration example. An analyst receives two tables:

- an **invoice** table with taxpayer identification numbers (`seller_tin`) and sales amounts;
- a **taxpayer** table with identification numbers, districts and economic sectors.

The goal is to combine them into one view of business activity.

3.1 Importing Pandas

```
import pandas as pd
```

Pandas exposes two main structures: the `Series` and the `DataFrame` [1]. A DataFrame is a two dimensional table with labelled rows and columns — close enough to a spreadsheet that it will feel familiar.

3.2 Creating the invoice table

```
invoices = pd.DataFrame({
    "seller_tin": ["101", "102", "103", "104"],
    "sales":      [500, 700, 900, 400]
})
print(invoices)
```

Index	seller_tin	sales
0	101	500
1	102	700
2	103	900
3	104	400

3.3 Creating the taxpayer table

```
taxpayers = pd.DataFrame({
    "seller_tin": ["101", "102", "103", "105"],
    "district":   ["Kigali", "Huye", "Rubavu", "Musanze"],
    "sector":     ["Retail", "Tech",
                   "Agriculture", "Restaurant"]
})
print(taxpayers)
```

Index	seller_tin	district	sector
0	101	Kigali	Retail
1	102	Huye	Tech
2	103	Rubavu	Agriculture
3	105	Musanze	Restaurant

3.4 How the two tables relate

Look at the keys carefully, because the rest of the article depends on this pattern:

- TINs 101, 102 and 103 appear in *both* tables.
- TIN 104 appears only in `invoices`.
- TIN 105 appears only in `taxpayers`.

Different join methods handle those last two cases very differently, and knowing which one you want is the whole game.

A note on data types

A taxpayer identification number is an identifier, not a quantity. Real TINs can contain leading zeros (e.g. `0123456789`), so they should be stored as strings rather than integers. You can confirm the storage type with:

```
print(invoices["seller_tin"].dtype)    # object
print(taxpayers["seller_tin"].dtype)   # object
```

If the two sides ever disagree — one `object`, one `int64` — `merge()` will silently return no matches. Check the dtype before you blame the join.

4. MERGE: Combining Tables on a Common Column

`pd.merge()` is the workhorse of dataset integration in Pandas. It combines two DataFrames by matching rows that share one or more key columns. Here, the key is `seller_tin` [3].

The general form is:

```
pd.merge(left, right, on="key", how="join_type")
```

The parameters that matter most are:

- `left`, `right` — the two DataFrames;
- `on` — the column(s) used to match rows;
- `how` — which unmatched rows to keep.

Four join styles cover most of the cases you will meet: inner, left, right and outer.

4.1 Inner join: keep only matches

An inner join keeps rows whose key exists on *both* sides.

When to use it

You want a clean analytic table: only taxpayers who have both an invoice and registered details. Anything present in just one table is out of scope.

```
result = pd.merge(invoices, taxpayers,
                  on="seller_tin", how="inner")
print(result)
```

seller_tin	sales	district	sector
101	500	Kigali	Retail
102	700	Huye	Tech
103	900	Rubavu	Agriculture

TINs 101, 102 and 103 matched and were joined. TIN 104 disappears because the taxpayer table has no record for it. TIN 105 disappears for the opposite reason: it is in the taxpayer table but not in the invoices.

Rule of thumb

`how="inner"` = intersection. Use it when downstream work needs complete information on every row.

4.2 Left join: keep everything from the left table

A left join preserves every row of the *first* DataFrame, whether or not a matching partner exists in the second.

When to use it

You are auditing invoices and cannot afford to lose any invoice, even one whose seller is not yet registered. The invoice side must stay complete.

```
result = pd.merge(invoices, taxpayers,
                  on="seller_tin", how="left")
print(result)
```

seller_tin	sales	district	sector
101	500	Kigali	Retail
102	700	Huye	Tech
103	900	Rubavu	Agriculture
104	400	NaN	NaN

All four invoices survive. TIN 104 has no taxpayer record, so `district` and `sector` are filled with `NaN` — Pandas's marker for a missing value.

Rule of thumb

`how="left"` = keep the left side whole; the right side contributes only where it can.

4.3 Right join: keep everything from the right table

A right join is the mirror image: every row of the *second* DataFrame survives.

When to use it

You are working from the taxpayer registration database and want to see which registered taxpayers actually filed invoices. Every registered taxpayer must remain visible.

```
result = pd.merge(invoices, taxpayers,
                  on="seller_tin", how="right")
print(result)
```

seller_tin	sales	district	sector
101	500	Kigali	Retail
102	700	Huye	Tech
103	900	Rubavu	Agriculture
105	NaN	Musanze	Restaurant

TIN 105 is kept even though it has no invoice; its `sales` value is `NaN`.

Rule of thumb

`how="right"` = keep the right side whole. A useful mental trick is that `right` is just `left` with the DataFrames swapped.

4.4 Outer join: keep everything from both tables

An outer join keeps every row from both sides.

When to use it

You are doing a data-quality review: which taxpayers have no invoices, and which invoices have no registered taxpayer? You want to see both kinds of orphan in the same result.

```
result = pd.merge(invoices, taxpayers,
                  on="seller_tin", how="outer")
print(result)
```

seller_tin	sales	district	sector
101	500	Kigali	Retail
102	700	Huye	Tech
103	900	Rubavu	Agriculture
104	400	NaN	NaN
105	NaN	Musanze	Restaurant

Every distinct TIN appears once, matched where possible and padded with `NaN` where not.

Rule of thumb

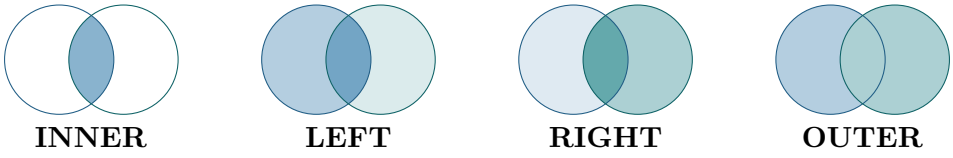
`how="outer"` = union of both sides. Ideal for reconciliation and orphan detection.

4.5 Comparing the four joins

Join type	Rows retained
`inner`	Rows whose key exists in both tables.
`left`	All rows from the first table, plus matches from the second.
`right`	All rows from the second table, plus matches from the first.
`outer`	All rows from both tables.

4.6 A picture of the four joins

Two overlapping circles capture the whole idea. The left circle is the first DataFrame, the right circle is the second, and the shaded region is what survives the join.



Blue: left dataset Teal: right dataset

One-line mnemonic

INNER = intersection • LEFT = all of the left • RIGHT = all of the right • OUTER = both.

4.7 A caution about duplicate keys

A common assumption is that merging n rows with m rows produces one row per input. That is only true when the join key is unique on at least one side. If a taxpayer appears twice in the registration table, every invoice for that taxpayer will match *both* registration rows, and you will get duplicate output rows.

Before merging on an identifier, check that it is what you think it is:

```
# How many duplicate TINs exist?
print(taxpayers["seller_tin"].duplicated().sum())

# How many distinct TINs exist?
print(taxpayers["seller_tin"].nunique())
```

Pandas can also enforce the relationship you expect via `validate`:

```
result = pd.merge(invoices, taxpayers,
                  on="seller_tin", how="left",
                  validate="many_to_one")
```

Here `many_to_one` says: many invoices may point to a single taxpayer, but never the other way around. On financial or administrative data, that one extra keyword has saved more analysts than almost any other Pandas feature.

5. CONCAT: Stacking Tables by Rows or Columns

Where `merge()` matches rows on a key, `concat()` simply glues DataFrames together along a chosen axis. It is the tool of choice when monthly files arrive in the same shape, or when a wide table has been split into pieces [4].

```
pd.concat([df1, df2], axis=0)
```

The single most important argument is `axis`:

- `axis=0` — stack rows (vertical).
- `axis=1` — glue columns side by side (horizontal).

5.1 Stacking rows: axis=0

Suppose January and February invoices arrived as two files with identical columns.

```
january = pd.DataFrame({
    "seller_tin": ["101", "102"],
    "sales":      [500, 700]
})
february = pd.DataFrame({
    "seller_tin": ["103", "104"],
    "sales":      [900, 400]
})

all_invoices = pd.concat(
    [january, february],
    axis=0,
    ignore_index=True
)
print(all_invoices)
```

Index	seller_tin	sales
0	101	500
1	102	700
2	103	900
3	104	400

February's two rows sit under January's, giving four rows in total. The `ignore_index=True` argument is what produces the fresh `0..3` index; without it, you would see two rows labelled `0` and two labelled `1`.

Remember

`axis=0` is the default for `pd.concat()`. So `pd.concat([a, b])` and `pd.concat([a, b], axis=0)` are identical.

5.2 Gluing columns: axis=1

Sometimes you hold two tables that already agree on row order and share the same index. Then you can simply paste the columns of one beside the other.

```
sales = pd.DataFrame({
    "seller_tin": ["101", "102", "103"],
    "sales":      [500, 700, 900]
})
details = pd.DataFrame({
    "district": ["Kigali", "Huye", "Rubavu"],
    "sector":   ["Retail", "Tech", "Agriculture"]
})

combined = pd.concat([sales, details], axis=1)
print(combined)
```

Index	TIN	Sales	District	Sector
0	101	500	Kigali	Retail
1	102	700	Huye	Tech
2	103	900	Rubavu	Agriculture

Unlike `merge()`, this call does *not* look at `seller_tin`. It aligns rows purely by index. If the two tables happen to list their taxpayers in different orders, the columns will be glued to the wrong rows — silently. When rows must be matched by an identifier, reach for `merge()` instead.

Silent misalignment is the classic bug

`pd.concat(axis=1)` respects index labels, not row positions. If the indexes do not agree, the result will contain `NaN`s (see the next subsection) and misplaced values. If you actually wanted to match on `seller_tin`, this is not the operation you meant.

5.2.1 What if `merge()` had been used instead?

The same two tables can be combined with `merge()`, but the result is strikingly different — and that difference is exactly why `merge()` is the safer tool for real work, provided you call it correctly.

Matching on the index. To make `merge()` behave like `concat(axis=1)`, you have to say so explicitly:

```
result = left.merge(right,
                    left_index=True,
                    right_index=True)
print(result)
```

Index	sales	district
1	700	Kigali

With no `how` argument, `merge()` defaults to an *inner* join, so only index label 1 survives. Rows 0 and 2 are silently dropped — perfectly correct behaviour for a merge, and almost never what you meant. To reproduce what `concat(axis=1)` did by default, you must remember to ask for it:

```
result = left.merge(right,
                    left_index=True,
                    right_index=True,
                    how="outer")
```

Index	sales	district
0	500	NaN
1	700	Kigali
2	NaN	Huye

Now the two results match. But note the trap: the "safe" version of `concat(axis=1)` — the one that does not silently drop rows — is the version where you had to *remember to add an argument*. That is precisely the kind of fragility that `merge()` on a real key avoids.

Matching on a real key. In practice the tables we actually care about share a column like `seller_tin`, not an integer row number. This is where `merge()` really earns its place — and where `concat(axis=1)` becomes dangerous.

```
invoices = pd.DataFrame({
    "seller_tin": ["101", "102", "103"],
    "sales":      [500, 700, 900]
})
details = pd.DataFrame({
    "seller_tin": ["103", "101", "102"],
    "district":   ["Rubavu", "Kigali", "Huye"]
})

merged = pd.merge(invoices, details, on="seller_tin")
print(merged)
```

Index	seller_tin	sales	district
0	101	500	Kigali
1	102	700	Huye
2	103	900	Rubavu

Note that `details` lists its TINs in a completely different order from `invoices`. That does not matter to `merge()`: it matches on the *values* in `seller_tin`, not on row positions.

Now compare with what `concat(axis=1)` would have done on the same two tables:

```
messy = pd.concat([invoices, details], axis=1)
print(messy)
```

Index	seller_tin	sales	seller_tin	district
0	101	500	103	Rubavu
1	102	700	101	Kigali
2	103	900	102	Huye

Look at the pairing. Taxpayer 101 has been glued to district `Rubavu` — which belongs to taxpayer 103. Every row is analytically wrong, yet the code ran without a single warning. The only visible clue is that duplicate `seller_tin` column header, which on a wider table you might not even notice.

The lesson in one line

`concat(axis=1)` trusts that the row orders of your DataFrames already agree. `merge()` verifies that the identifier values do. When the two tables really describe the same entities, the second check is the one you can rely on.

6. JOIN: Combining DataFrames Using Their Indexes

Pandas also exposes a DataFrame method called `join()`. At first glance it looks like `merge()`, but it behaves differently by default: it combines DataFrames using their *indexes*, not ordinary columns [5].

6.1 What a DataFrame index actually is

Every DataFrame carries an index — the labels down the left column. By default they are just the row numbers:

Index	TIN	Sales
0	101	500
1	102	700
2	103	900

But you can promote a meaningful identifier to be the index, which is often what you want when data represents a set of entities.

6.2 A working join

```
invoices = pd.DataFrame({
    "seller_tin": ["101", "102", "103"],
    "sales":      [500, 700, 900]
})
taxpayers = pd.DataFrame({
    "district": ["Kigali", "Huye", "Rubavu"],
    "sector":   ["Retail", "Tech", "Agriculture"]
}, index=["101", "102", "103"])

invoices = invoices.set_index("seller_tin")
result   = invoices.join(taxpayers, how="left")
print(result)
```

seller_tin	sales	district	sector
101	500	Kigali	Retail
102	700	Huye	Tech
103	900	Rubavu	Agriculture

There are three steps to notice. First, `set_index("seller_tin")` makes the TIN the row label of the invoice table. Second, the taxpayer table was already constructed with those same TINs as its index. Third, `join()` matches on the labels.

6.3 Join types

`join()` accepts the same four `how` values as `merge()`:

```
invoices.join(taxpayers, how="inner")  # matching indexes
invoices.join(taxpayers, how="left")   # every invoice index
invoices.join(taxpayers, how="right")  # every taxpayer index
invoices.join(taxpayers, how="outer")  # every index on both sides
```

The semantics are exactly the same as in Section 4. The only difference is that the matching is on the index rather than a specified column.

MERGE vs. JOIN vs. CONCAT

Operation	What it matches on
`merge()`	Common columns you name with `on`.
`join()`	The two DataFrames' indexes.
`concat()`	Nothing — it just stacks by axis.

7. Adding New Columns

Not every task is a combination of two tables. Very often you need to create a new column inside an existing one — a tax amount, a rate, a category. Pandas offers four idioms, and knowing which one to reach for is often the whole trick.

7.1 Direct assignment

The simplest form:

```
df["tax_rate"] = 0.18
```

TIN	Sales	Tax rate
101	500	0.18
102	700	0.18
103	900	0.18

Use this when the same value applies to every row.

7.2 Calculated assignment

More often the new column depends on existing ones:

```
df["tax_amount"] = df["sales"] * 0.18
```

TIN	Sales	Tax amount
101	500	90
102	700	126
103	900	162

Note that there is no explicit loop. Pandas applies the multiplication to every row at once — this is the "vectorised" style of working that makes DataFrame code so much faster than writing Python `for` loops over rows [1, 2].

A caveat about the 18% rate

The rate in this example is illustrative. Real tax calculations depend on the applicable rules, the category of the good or service, and the exemptions available. This article teaches the *programming* operation, not a tax-compliance rule.

7.3 Using `assign()`

`assign()` returns a *new* DataFrame with the extra column, leaving the original untouched. It is convenient when you want to chain several transformations:

```
df = df.assign(
    tax_rate   = 0.18,
    tax_amount = df["sales"] * 0.18
)
```

Both columns are created in a single call.

7.4 Using `insert()`

`insert()` is the only one of the four that lets you choose *where* the new column lands:

```
df.insert(1, "tax_rate", 0.18)
```

If the original columns were `seller_tin` and `sales`, the new order becomes:

seller_tin	tax_rate	sales
101	0.18	500
102	0.18	700
103	0.18	900

Use `insert()` when column order matters (e.g. for a fixed report layout). Use assignment or `assign()` the rest of the time.

Inserting a calculated column. `insert()` is not limited to constants. Its third argument accepts any array-like object that has one value per row — a list, a NumPy array, or (most usefully) the result of an expression over an existing column. So you can insert a *calculated* column at a specific position in exactly the same call:

```
df.insert(2, "tax_amount", df["sales"] * 0.18)
```

If `df` previously had the columns `seller_tin`, `tax_rate` and `sales`, the new order becomes:

seller_tin	tax_rate	tax_amount	sales
101	0.18	90	500
102	0.18	126	700
103	0.18	162	900

Notice what did *not* change. The multiplication was evaluated *before* `insert()` ran, producing a `Series` with one value per row; `insert()` then slotted that Series into position 2. The existing columns `tax_rate` and `sales` were pushed right by one place, and their data was untouched.

Three rules for calculated inserts

1. **Length must match.** `insert()` raises a `ValueError` if the value array has a different number of rows than the DataFrame. A scalar like `0.18` is the exception — it is broadcast to every row.

2. **Index alignment matters.** If you pass a `Series` whose index differs from the DataFrame's, Pandas aligns by label and silently produces `NaN` wherever labels do not match. The safest pattern is to compute the new column *from the same DataFrame* you are inserting into (as in `df["sales"] * 0.18`), which guarantees matching indexes.

3. **No duplicate names.** Unlike assignment (`df["col"] = ...`), which overwrites an existing column, `insert()` raises `ValueError` if the name already exists. That is a feature when you want a typo to fail loudly, and a nuisance when you are iterating — in that case, drop the old column first.

Choosing between `insert()` and `assign()`. When the new column is calculated and you only care that it exists, `assign()` is usually cleaner because it returns a new DataFrame you can chain from:

```
# assign: returns a new DataFrame
df = df.assign(tax_amount=df["sales"] * 0.18)
```

Reach for `insert()` specifically when the *column position* matters — a fixed report layout, a CSV exported to a downstream system that reads columns by index, or a visual ordering you want to preserve. Also remember that `insert()` modifies the DataFrame *in place* and returns `None`, so you cannot chain it:

```
# This does NOT work -- df.insert(...) returns None
df = df.insert(2, "tax_amount", df["sales"] * 0.18)
```

Call it as a statement on its own line, and the DataFrame it belongs to will be updated as a side effect.

7.5 Quick comparison

Method	Typical form	Best for
Assignment	`df["new"] = ...`	One-off, simple.
`assign()`	`df = df.assign(new=...)`	Chained transformations.
`insert()`	`df.insert(i, "new", ...)`	Controlling column position.

8. Adding New Rows

Rows arrive after the fact too: a new taxpayer registers, a late invoice lands, a correction needs appending. There are two clean ways to handle it.

8.1 A single new row with `loc`

```
df = pd.DataFrame({
    "seller_tin": ["101", "102"],
    "sales":      [500, 700]
})

df.loc[len(df)] = ["103", 900]
print(df)
```

Index	seller_tin	sales
0	101	500
1	102	700
2	103	900

`len(df)` returns the current row count, so the new row gets the next integer label. That trick only works on a zero-based, consecutive index.

8.2 Multiple rows with `concat()`

For more than one row, prepare a small DataFrame and stack:

```
new_records = pd.DataFrame({
    "seller_tin": ["103", "104"],
    "sales":      [900, 400]
})

df = pd.concat([df, new_records], ignore_index=True)
print(df)
```

Index	seller_tin	sales
0	101	500
1	102	700
2	103	900
3	104	400

Professional practice

For multiple additions, always prefer building a new DataFrame and calling `pd.concat()`. Inserting rows one by one in a loop is very slow on large datasets — every call copies memory — and the historical `DataFrame.append()` method that used to do this has been removed from modern Pandas.

9. GROUPBY: Grouping and Summarising

The last major building block is grouping. `groupby()` splits rows into buckets that share a value, then applies a function to each bucket and combines the results. Pandas documentation calls this the split–apply–combine pattern [6].

The most common aggregations after a group are:

- `sum()` — total.
- `mean()` — average.
- `count()` — non-missing values.
- `nunique()` — distinct values.
- `max()` / `min()` — extremes.

9.1 A small example

```
import pandas as pd

data = pd.DataFrame({
    "seller_tin": ["101", "101", "102", "102", "101"],
    "district":   ["Kigali", "Kigali", "Huye", "Huye", "Kigali"],
    "sales":      [500, 300, 700, 200, 400]
})
```

Three invoices for taxpayer 101, two for taxpayer 102.

9.2 Total sales per taxpayer

```
result = data.groupby("seller_tin")["sales"].sum()
print(result)
```

```
seller_tin
101    1200
102     900
Name: sales, dtype: int64
```

To see the arithmetic: $500 + 300 + 400 = 1200$ for TIN 101, and $700 + 200 = 900$ for TIN 102.

9.3 Grouping by more than one column

```
result = (
    data.groupby(["seller_tin", "district"])["sales"]
        .sum()
        .reset_index()
)
print(result)
```

seller_tin	district	sales
101	Kigali	1200
102	Huye	900

Pandas creates one bucket per distinct combination of taxpayer and district.

9.4 Why `reset_index()` matters

After a group operation, the grouping key becomes the *index* of the result. That is fine for inspection, but awkward the moment you want to merge the summary back into another table or export it to a CSV. `reset_index()` turns the index back into an ordinary column:

```
# Key stays in the index
data.groupby("seller_tin")["sales"].sum()

# Key becomes a normal column
data.groupby("seller_tin")["sales"].sum().reset_index()
```

Prefer the second form whenever the grouped result is going to feed anything downstream.

9.5 A handful of other aggregations

```
data.groupby("seller_tin")["sales"].sum()        # total
data.groupby("seller_tin")["sales"].mean()       # average
data.groupby("seller_tin")["sales"].count()      # non-missing
data.groupby("seller_tin")["sales"].max()        # highest
data.groupby("seller_tin")["sales"].min()        # lowest
data.groupby("seller_tin")["district"].nunique()  # distinct districts
```

These are the building blocks of any summary report: sector totals, monthly activity, per-taxpayer indicators, and so on.

10. Other Frequently Used Operations

The following methods come up constantly alongside merging, joining and grouping. Knowing them by sight makes Pandas code much easier to read.

Operation	Example	Purpose
`head()`	`df.head(5)`	First five rows.
`tail()`	`df.tail(5)`	Last five rows.
`shape`	`df.shape`	(rows, columns).
`len()`	`len(df)`	Number of rows.
`count()`	`df["sales"].count()`	Non-missing values.
`nunique()`	`df["seller_tin"].nunique()`	Distinct values.
`sum()`	`df["sales"].sum()`	Total.
`mean()`	`df["sales"].mean()`	Average.
`max()`	`df["sales"].max()`	Maximum.
`min()`	`df["sales"].min()`	Minimum.
`reset_index()`	`df.reset_index()`	Index back as a column.
`set_index()`	`df.set_index("seller_tin")`	Column becomes the index.
`sort_values()`	`df.sort_values("sales")`	Sort by a column.
`drop_duplicates()`	`df.drop_duplicates()`	Remove repeated rows.
`isna()`	`df.isna()`	Locate missing values.
`fillna()`	`df.fillna(0)`	Replace missing values.
`assign()`	`df.assign(rate=0.18)`	Create columns.
`insert()`	`df.insert(1,"rate",0.18)`	Insert column at position.
`merge()`	`pd.merge(df1,df2)`	Combine on common columns.
`concat()`	`pd.concat([df1,df2])`	Stack DataFrames.

11. Comparing the Main Combination Operations

Operation	Purpose	Direction
`df["new"]=values`	Add a column.	Horizontal.
`assign()`	Add one or more columns.	Horizontal.
`insert()`	Add a column at a fixed position.	Horizontal.
`merge()`	Combine on matching keys.	Horizontal.
`join()`	Combine on indexes.	Horizontal.
`concat(axis=1)`	Glue columns by index alignment.	Horizontal.
`loc[]`	Assign a single new row.	Vertical.
`concat(axis=0)`	Stack rows.	Vertical.
`groupby()`	Group + aggregate.	Summarisation.

The one distinction to remember under exam pressure

1. Adding a *column*? Assignment, `assign()`, `insert()`, `merge()`, `join()` or `concat(axis=1)` — pick based on whether you are matching on a key or just pasting.
2. Adding a *row*? `loc[]` for a single row, `concat(axis=0)` for many.
3. *Summarising*? `groupby()` plus an aggregation.

These three buckets never mix. Identify the bucket first and the method choice becomes obvious.

12. End-to-End Example: Monthly EBM Analysis

Now put the pieces together. Suppose an analyst receives a log of electronic invoices with these columns:

- taxpayer identification number;
- invoice number;
- invoice date;
- invoice status;
- total invoice amount in RWF.

The task: produce one row per taxpayer per month, showing the total of *completed* invoices.

12.1 Step 1 — keep only completed invoices

Cancelled or pending invoices do not belong in a completed-sales total.

```
completed = df[df["invoice_status"] == "Completed"]
```

12.2 Step 2 — derive the invoice month

The date column has full dates; monthly reporting needs just the year and month.

```
completed = completed.assign(
    month = completed["invoice_date"].dt.to_period("M")
)
```

Original date	Extracted month
2026-01-05	2026-01
2026-01-18	2026-01
2026-02-10	2026-02

The `.dt` accessor unlocks date properties, and `to_period("M")` collapses a date to its month period.

12.3 Step 3 — group by taxpayer and month

```
monthly_ebm = (
    completed
    .groupby(["seller_tin", "month"])["total_amount_rwf"]
    .sum()
    .reset_index()
)
```

Four things happen in that chain: group, select the amount column, sum, and put the grouping keys back as ordinary columns.

12.4 Step 4 — inspect the result

```
print("Monthly rows:", len(monthly_ebm))
print("Unique taxpayers:",
      monthly_ebm["seller_tin"].nunique())
print(monthly_ebm.head(5).to_string(index=False))
```

Illustrative output

seller_tin	month	total_amount_rwf
101	2026-01	1500000
101	2026-02	1800000
102	2026-01	2300000
103	2026-01	1200000
103	2026-02	1600000

Values are illustrative and do not represent real administration records.

12.5 Step 5 — enrich with registration data

The monthly summary is more useful with district and sector attached. One `merge()` does it:

```
monthly_ebm = pd.merge(
    monthly_ebm,
    taxpayer_table,
    on="seller_tin",
    how="left"
)
```

A *left* join is correct here: we want every taxpayer-month row we computed, and we accept `NaN` for the rare seller whose registration is missing.

12.6 What the analyst can do next

The result supports all the usual work of a tax analyst: compare monthly sales across taxpayers, spot unusual swings, group by sector or district, and build monitoring indicators. The pipeline (filter → derive → group → enrich) is exactly the shape of most real-world data preparation.

13. Common Mistakes to Avoid

Knowing the commands is not the same as using them well. The following traps catch almost every beginner at least once.

13.1 Using `concat()` when you meant `merge()`

```
pd.concat([invoices, taxpayers], axis=1)   # often wrong!
```

This does not match by TIN. It glues columns by index label, which will misalign the moment the row orders differ. **Fix:** if the relationship is defined by an identifier, use `merge()` on that identifier.

13.2 Choosing the wrong join type

An inner join silently drops invoices whose seller is not registered. A left join silently drops taxpayers who have not yet filed. Either can look correct on the surface and be very wrong analytically. **Fix:** before running the merge, decide which side must remain complete and choose `how` to match.

13.3 Treating `NaN` as zero

A missing sales figure is not a zero. It usually means the information is *unavailable*. Replacing it with `0` will understate activity for that taxpayer. **Fix:** investigate why values are missing before deciding how to fill them.

13.4 Ignoring duplicate keys

Two registration rows for the same TIN will duplicate every invoice for that taxpayer. Totals then overstate activity. **Fix:** run the `duplicated().sum()` check and use `validate="many_to_one"` on the merge.

13.5 Confusing `count()` and `size()`

Both are often used after a `groupby`, but they mean different things:

```
# Non-missing values in the column
df.groupby("seller_tin")["sales"].count()

# Total rows in the group (missing included)
df.groupby("seller_tin").size()
```

If `sales` contains `NaN`, the two results will differ by exactly the count of those missing values. That difference shows up in invoice counts and averages — choose deliberately.

13.6 Forgetting index alignment in `concat(axis=1)`

Two DataFrames with the same number of rows are *not* interchangeable by row position. Pandas aligns on index labels, and if the labels disagree, you get `NaN` and misplaced values. **Fix:** inspect `df.index` before horizontal concatenation, or use `merge()` on a real key.

14. Practice and Self-Assessment

14.1 Multiple-choice questions

Q1. You have January and February invoices in two files with identical columns. You want February's rows placed below January's. Which call is right?

A. `pd.merge(jan, feb)`

B. `pd.concat([jan, feb], axis=0)`

C. `pd.concat([jan, feb], axis=1)`

D. `jan.groupby(...)`

Answer: B. `pd.concat([jan, feb], axis=0)` stacks the two DataFrames vertically, placing February's rows below January's.

Why not A? `pd.merge()` matches rows on columns or keys. It is not the right tool when you simply want to append records that already share the same structure.

Why not C? `axis=1` would glue the columns side by side, not stack the rows.

Q2. You want to attach taxpayer details to invoices using the `seller_tin` column. Which method?

A. `pd.merge(invoices, taxpayers, on="seller_tin")`

B. `pd.concat([invoices, taxpayers], axis=1)`

C. `invoices.insert(1, "district", ...)`

D. `invoices.head()`

Answer: A. `pd.merge()` combines DataFrames on a shared key, such as `seller_tin`, so each invoice is matched to its corresponding taxpayer row.

Why not B? This is the classic trap. `pd.concat(axis=1)` does add columns side by side, but it aligns rows by *index label*, not by `seller_tin`. If the two DataFrames were loaded in a different row order — or have different integer indexes — the district and sector are glued to the wrong invoices, with no error and no warning. Only `merge()` honours `seller_tin` as a key.

Why not C? `insert()` adds a single column to *one* DataFrame. It cannot pull values in from a second DataFrame, so it cannot match taxpayer details to invoices.

Q3. Using `pd.merge(invoices, taxpayers, on="seller_tin")`, you must keep every invoice — even where the seller is unregistered. Which `how` value should you choose?

A. `"inner"`

B. `"right"`

C. `"left"`

D. `"outer"`

Answer: C. Left. A left join retains every row from the *first* DataFrame (`invoices`), padding missing taxpayer fields with `NaN`.

Why not A? An inner join keeps only invoices whose `seller_tin` also exists in the taxpayer table. Any invoice from an unregistered seller would silently vanish.

Why not B? A right join preserves the *second* DataFrame (`taxpayers`). Invoices belonging to unregistered sellers would still be dropped.

Q4. What does `how="outer"` do in a merge?

A. Keeps only matching rows.

B. Keeps all rows from both sides.

C. Keeps only the first DataFrame.

D. Keeps only the second DataFrame.

Answer: B. An outer join retains every row from both DataFrames, joining matches where possible and leaving `NaN` where not.

Why not A? "Keeps only matches" describes an *inner* join. "Keeps only the first" is a *left* join; "only the second" is a *right* join.

Q5. Which expression creates a new column `tax` containing 18% of each value in `sales`?

A. `df.groupby("sales").sum()`

B. `df["tax"] = df["sales"] * 0.18`

C. `pd.concat([df, df], axis=1)`

D. `df.reset_index()`

Answer: B. Multiplying each sales value by 0.18 produces 18% of that value, and assigning the result to `df["tax"]` stores it as a new column.

Why not A? `groupby("sales").sum()` returns a summary Series grouped by sales value; it does not add a `tax` column to the original DataFrame.

Why not C? `pd.concat(axis=1)` does add columns — but only by pasting *existing* columns side by side. It performs no calculation, so it cannot produce 18% of anything.

Q6. What does `pd.concat([df1, df2], axis=1)` do?

A. Adds rows.

B. Groups rows.

C. Adds columns aligned by index.

D. Removes duplicate rows.

Answer: C. `axis=1` combines DataFrames horizontally, placing their columns side by side and aligning rows by index label.

Why not A? Adding rows is what `pd.concat([...], axis=0)` does, not `axis=1`.

Q7. Which DataFrame *method* combines two DataFrames using their indexes *by default*?

A. `join()`

B. `groupby()`

C. `nunique()`

D. `count()`

Answer: A. `DataFrame.join()` aligns rows on the index unless you tell it otherwise with `on=`. `merge()` *can* also match indexes, but only if you explicitly pass `left_index=True` and `right_index=True`; matching on indexes is not its default.

Why not B? `groupby()` groups rows by column values or index levels within a *single* DataFrame. It does not combine two DataFrames.

14.2 Practical exercise

```python
import pandas as pd

invoices = pd.DataFrame({
    "seller_tin": ["101", "102", "103", "104"],
    "sales":      [500, 700, 900, 400]
})

taxpayers = pd.DataFrame({
    "seller_tin": ["101", "102", "103", "105"],
    "district":   ["Kigali", "Huye", "Rubavu", "Musanze"],
    "sector":     ["Retail", "Tech",
                   "Agriculture", "Restaurant"]
})
```

1. Inner-join the two tables.
2. Left-join, then list the invoice rows with no match.
3. Right-join, then list the taxpayers with no invoice.
4. Outer-join and confirm all rows appear.
5. Add a `tax_rate` column of 0.18.
6. Add a `tax_amount` column = sales × rate.
7. Create a small DataFrame of two more taxpayers and stack it below `invoices`. Inspect what happens to `tax_rate` and `tax_amount` for the new rows, and explain the result.
8. Total sales per taxpayer with `groupby()`.
9. Count the number of distinct taxpayers.

14.3 Suggested answers

```python
# 1. Inner join
inner = pd.merge(invoices, taxpayers,
                 on="seller_tin", how="inner")

# 2. Left join + orphans
```

```python
left = pd.merge(invoices, taxpayers,
                on="seller_tin", how="left")
orphan_invoices = left[left["district"].isna()]

# 3. Right join + orphans
right = pd.merge(invoices, taxpayers,
                 on="seller_tin", how="right")
orphan_taxpayers = right[right["sales"].isna()]

# 4. Outer join
outer = pd.merge(invoices, taxpayers,
                 on="seller_tin", how="outer")

# 5-6. Tax columns
invoices["tax_rate"]   = 0.18
invoices["tax_amount"] = invoices["sales"] * invoices["tax_rate"]

# 7. Stack new records
new_records = pd.DataFrame({
    "seller_tin": ["106", "107"],
    "sales":      [800, 1000]
})
combined = pd.concat([invoices, new_records], ignore_index=True)
# tax_rate and tax_amount become NaN for 106 and 107 because
# pd.concat(axis=0) aligns by column name and the new rows
# supply no values for those two columns.

# 8. Total per taxpayer
summary = combined.groupby("seller_tin")["sales"].sum()

# 9. Distinct taxpayers
unique_taxpayers = combined["seller_tin"].nunique()
print(summary)
print("Unique taxpayers:", unique_taxpayers)
```

Why not `merge()` or `join()` on 7th question ? Both are *matching* operations. They assume a relationship between entities — an invoice that belongs to a taxpayer, an order that belongs to a customer — and their job is to *enrich* existing rows with columns from a second table. Neither is designed to append observations. Step 7 is a *vertical* task: the new rows are simply additional invoices, not related to the existing ones by any key. The correct tool is therefore `concat(axis=0)`.

Before trying any of the alternatives, look at the two tables we are working with. After steps 5 and 6, `invoices` holds four rows and four columns:

Index	seller_tin	sales	tax_rate	tax_amount
0	101	500	0.18	90
1	102	700	0.18	126
2	103	900	0.18	162
3	104	400	0.18	72

and `new_records`, freshly constructed in step 7, has two rows and only two columns — it knows nothing about the tax columns that were added later:

Index	seller_tin	sales
0	106	800
1	107	1000

These two frames are *slices of the same invoice log*, recorded at different times. Nothing in either one looks up anything in the other. Now compare what each operation actually produces.

It is still instructive to see what would happen if you reached for `merge()` instead:

```
# Wrong tool: merge on the same key as the joins above
bad = pd.merge(invoices, new_records,
               on="seller_tin", how="inner")
# -> empty DataFrame
```

The result is not just imperfect — it is *empty*:

seller_tin	sales_x	tax_rate	tax_amount	sales_y
(0 rows)				

Two things are visible even in that header. First, neither new TIN (`106`, `107`) exists in the invoice table, so no match is possible and every key disappears. Second, the column names have been mangled: `sales` from the left table became `sales_x` and `sales` from the right table became `sales_y`, because Pandas saw the same name on both sides and silently disambiguated.

With `how="left"`, the result is not empty — but it is still wrong:

seller_tin	sales_x	tax_rate	tax_amount	sales_y
101	500	0.18	90	NaN
102	700	0.18	126	NaN
103	900	0.18	162	NaN
104	400	0.18	72	NaN

Four rows, exactly what we started with. *None* of the new records survived. The left join faithfully kept the left side whole — as it was designed to do — and then treated `new_records` as a lookup table from which to draw extra columns. Since no row in it matched, every lookup came back empty and two invoices silently disappeared. No error, no warning, just missing data.

Only `how="outer"` retains all six rows. But look at the shape it forces on them:

seller_tin	sales_x	tax_rate	tax_amount	sales_y
101	500	0.18	90	NaN
102	700	0.18	126	NaN
103	900	0.18	162	NaN
104	400	0.18	72	NaN
106	NaN	NaN	NaN	800
107	NaN	NaN	NaN	1000

Every record now carries its sales value in one of two different columns depending on its origin. A value that used to be a single, clean `sales` column is smeared across `sales_x` and `sales_y`. Any downstream code written against `df["sales"]` — a `sum()`, a `mean()`, a chart, an export — breaks immediately, because that column no longer exists. And the two new invoices carry no tax information at all, simply because `new_records` never had those columns. The merge did nothing wrong; it was simply the wrong tool.

Compare with the correct operation:

```
combined = pd.concat([invoices, new_records],
                     ignore_index=True)
```

Index	seller_tin	sales	tax_rate	tax_amount
0	101	500	0.18	90
1	102	700	0.18	126
2	103	900	0.18	162
3	104	400	0.18	72
4	106	800	NaN	NaN
5	107	1000	NaN	NaN

Six rows, four columns, no renaming, and the missing tax figures correctly shown as `NaN`. That `NaN` is not a defect — it is the true record: those two invoices were logged *before* the tax columns existed, so no tax figure was ever computed for them. An analyst reading the table knows exactly what happened and why.

But relying on `merge(how="outer")` to make appending "work" is fragile and misleading:

- **One wrong keyword away from data loss.** A maintainer who changes `"outer"` to `"inner"` — a change that is harmless in most merge contexts — would silently delete every newly appended record.
- **Wrong semantics.** `merge()` implies that the two tables describe the same entities from different angles and need reconciling. They do not. They are slices of the same log recorded at different times.
- **Duplicated keys become cartesian products.** If the new records happen to reuse an existing TIN — a corrected invoice, a re-issued sale — `merge()` will pair each new record against every existing invoice for that TIN.

The duplicate-key trap, seen concretely. That last bullet is worth one small worked example. Suppose the original log already contained *two* invoices for TIN 101, and the correction file contained *one* replacement row for the same taxpayer:

invoices (extract)

seller_tin	sales	tax_rate	tax_amount
101	500	0.18	90
101	300	0.18	54
102	700	0.18	126

new_records (extract)

seller_tin	sales
101	550

A merge on `seller_tin` with `how="outer"` produces:

seller_tin	sales_x	tax_rate	tax_amount	sales_y
101	500	0.18	90	550
101	300	0.18	54	550
102	700	0.18	126	NaN

The single correction `550` has been duplicated: it now appears against *both* original invoices for TIN 101, as if each had independently been revised to the same value. That is almost never what

was intended. With `concat(axis=0)`, the three original rows plus the one new row remain four distinct observations — the correction is simply the fourth entry in the log, not a rewrite of the first three.

The same objections apply to `join()`, which is simply `merge()` expressed against the index. If you first ran `invoices.set_index("seller_tin")` and then `invoices.join(new_records.set_index("seller_t` you would be *matching* taxpayer identifiers, not *stacking* invoices — a different operation dressed up in similar clothes, and subject to every problem just described.

The rule behind the rule

`merge()` and `join()` are for *combining columns of related tables*. `concat(axis=0)` is for *appending observations to a log*. When a new row is a fresh observation rather than a lookup into another table, `concat` is the semantically correct choice — and the `NaN` that appears for the two new tax columns is a *feature* of that choice: it faithfully records that those rows were created *before* the tax columns existed, not that the append failed.

A method for tackling exam problems

For every scenario, ask the same four questions in order:

1. Am I adding *columns* or *rows*?
2. Does the relationship depend on an *identifier*, or is it purely positional?
3. Which side of the data must remain *complete*?
4. Am I *combining* observations, or *summarising* them?

Answer these and the Pandas method usually falls out automatically — even when the exam uses wording you have not seen.

15. Conclusion

Integrating data is one of the most common tasks — and one of the most common places to quietly introduce errors — in applied analysis. The tables you need are usually spread across several sources; combining them correctly without losing, duplicating or misaligning information is a skill, not a command to memorise.

`merge()` is the tool of choice when the two sides share a key column, such as a taxpayer identification number. `join()` is the same idea expressed in terms of indexes, and shines when you have deliberately set meaningful indexes on your DataFrames. `concat()` handles the simpler cases: stacking rows from same-shaped files, or gluing columns when row order is already guaranteed.

Direct assignment, `assign()` and `insert()` give you clean idioms for adding columns, while `loc[]` and `concat()` do the same for rows. Finally, `groupby()` turns detailed records into the summaries that reporting, monitoring and decision-making actually run on.

For an analyst working with EBM invoice records, these operations cover the whole arc of the job: bring invoices and registration details together, attach sector and district information, produce monthly summaries, and check that nothing was dropped along the way.

The broader lesson is that good data work is less about knowing Python commands than about understanding the structure of the data and the relationships between tables. Once those relationships are clear, Pandas becomes what it is meant to be — a tool for producing reliable,

reproducible and meaningful analysis.

References

[1] W. McKinney, *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[2] J. VanderPlas, *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol, CA, USA: O'Reilly Media, 2016.

[3] The pandas development team, "`pandas.merge`," *pandas Documentation*. [Online]. Available: https://pandas.pydata.org/docs/reference/api/pandas.merge.html. [Accessed: Nov. 11, 2025].

[4] The pandas development team, "`pandas.concat`," *pandas Documentation*. [Online]. Available: https://pandas.pydata.org/docs/reference/api/pandas.concat.html. [Accessed: Nov. 11, 2025].

[5] The pandas development team, "`pandas.DataFrame.join`," *pandas Documentation*. [Online]. Available: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.join.html. [Accessed: Nov. 11, 2025].

[6] The pandas development team, "Group by: split–apply–combine," *pandas Documentation*. [Online]. Available: https://pandas.pydata.org/docs/user_guide/groupby.html. [Accessed: Nov. 11, 2025].

[7] The pandas development team, "Indexing and selecting data," *pandas Documentation*. [Online]. Available: https://pandas.pydata.org/docs/user_guide/indexing.html. [Accessed: Nov. 11, 2025].